

TypeGo: An OS Runtime for Embodied Agents

Guojun Chen
guojun.chen@yale.edu
Yale University

Alex Schott
a.schott@yale.edu
Yale University

Lin Zhong
lin.zhong@yale.edu
Yale University

Abstract

Large language models (LLMs) can plan behavior for embodied agents from natural language, but treating the LLM as a request/response oracle on the critical path is fundamentally at odds with real-time control and concurrent goals. We argue for an *operating-system-style runtime* for embodied agents, and instantiate this idea in an early prototype, TypeGo. TypeGo gives each task its own planning stack and runs those stacks as asynchronous loops at multiple timescales, overlapping deliberation with execution. It manages the agent’s physical body like an OS manages hardware: the *Skill Kernel* arbitrates typed physical subsystems among concurrent per-task *processes*, a scheduler preempts them and resumes or replaces each by source, and *speculative skill streaming* hides LLM latency behind ongoing motion, while a fast first-action path yields visible feedback within a second. Users program behavior through natural language *prescriptions* that TypeGo dispatches to the LLM-based planners or compiles into low-latency interrupt handlers. Our prototype on Kalos, a Unitree Go2 quadruped, provides preliminary evidence for the design: in our current task suite, it cuts per-step delay by 50% over step-by-step planning and time-to-first-action by 73% over monolithic planning, while admitting concurrent tasks at low scheduling overhead.

1 Introduction

Robots today execute a rich catalog of low-level skills: quadrupeds walk, trot, climb stairs, and balance, while humanoids add grasping and manipulation on top; many platforms ship turnkey APIs for locomotion, speech, and perception. These skills solve basic motion and action well, but they do not tell a robot what to do when a human asks for something new, the scene changes unexpectedly, or several goals compete for attention. The hard problem has shifted from how a robot body *moves* to how an embodied system *decides*, and hard-coded task logic or per-task trained policies do not scale to the open world.

LLMs offer a path to close the decision-making gap: pretrained on large-scale data, they supply the common sense to interpret a situation and compose the robot’s existing skills into a plan [1, 2]. But using an LLM to drive an embodied agent raises three system challenges that persist even as models become faster and more capable.

(i) First, LLM inference is too slow for real-time reaction. There are systems that place a language or vision-language model in the perception-action loop for high-level reasoning to achieve impressive open-world generality [3–7]. But they operate far below real-time, with long pauses between consecutive actions and demonstrations commonly shown at increased playback speed. Small models respond fast but plan poorly, while state-of-the-art models are capable but large, adding seconds of latency. This latency-quality tradeoff is fundamental, and neither extreme alone suffices for real-time control. (ii) Second, a real-world embodied agent rarely runs

one task in isolation. It may patrol an area while communicating with a nearby human, and still needs to watch for hazards. Arbitrating the robot’s shared actuators among concurrent tasks while honoring their priorities and preempting cleanly are classic OS problems that the LLM literature has largely sidestepped. (iii) Third, one-shot natural-language instructions cover only goals. Embodied agents also need *reactive* rules (e.g., “back up if anything appears within 0.2 m”) that must fire far faster than an LLM call. Making natural language a first-class medium for both goals and fast reaction rules, authored by non-programmers without a code editor, remains an open question.

To tackle these challenges, we hypothesize that *LLM-based control of an embodied agent should run as a continuously executing, asynchronous runtime rather than a request/response loop around the LLM*. In effect, it works like a small OS for the agent. We build **TypeGo**, a prototype that realizes this runtime design. TypeGo runs a reflex layer (S0) and a process scheduler (S3) globally, plus a planning stack (S2/S1) instantiated once per running task, all over a *Skill Kernel* that mediates the body’s actuators. Several tasks therefore plan at once against one body, and the runtime must arbitrate, preempt, and resume them. TypeGo is orthogonal to work that accelerates a single LLM call. The design instead aims to *hide* LLM calls from the critical path through speculation and tiered dispatch, so any per-call speedup composes directly with its architecture.

This paper develops three mechanisms that make the runtime hypothesis concrete:

- **Speculative skill streaming (§6)**. TypeGo runs its planning layers as independent loops at distinct levels of abstraction: a fast reflex layer (S0), an action streamer (S1), slower task-decomposer (S2) and scheduler (S3). Specifically, the S1 features *speculative skill streaming*, which overlaps planning with skill execution and stores the upcoming skill calls in a bounded queue. As a result, the LLM planning latency is hidden behind ongoing motion while bounded look-ahead preserves per-step adaptivity in a dynamic environment.
- **OS-style runtime with semantic scheduling (§5)**. Each task creates a process with its own process control block (PCB) and its own S2/S1 planning pair. A *Skill Kernel* arbitrates the categorized robot’s resources for processes. S3 schedules these processes semantically rather than by fixed numeric priority. When two processes compete for the same resource, S3 decides which wins based on the scene and prompt guidelines. Whether a preempted process then resumes depends on its origin: a user task does not resume by default, whereas an S3-initiated task resumes automatically.
- **Natural language as a first-class programming medium (§4)**. Users author *prescriptions* of three kinds: natural-language *tasks* that specify goals, a *persona* that specifies behavior traits, and *reflexes* that specify reactive rules. Tasks are supplied online to the S1–S3 planners; the persona compiles offline into per-layer

guidelines that bias S1–S3 planning; reflexes compile into Python condition/action functions that S0 evaluates at high frequency. Together, they let non-programmers author deliberate, standing, and reactive behavior without writing code.

We report an early implementation of TypeGo (§7) and use a prototype deployment on a Unitree Go2 quadruped to validate the design direction in single- and concurrent-task scenarios (§8). In our current task suite, TypeGo cuts per-step delay by 50% over step-by-step planning and time-to-first-action (TTFA) by 73% over monolithic planning, while adding low coordination overhead: about 200 ms for S3 scheduling and 50 ms for S0 reflex handling, plus 1.2 s of replanning when S3 switches between tasks.

2 Related Work

Hierarchical robot control. TypeGo’s planner hierarchy follows the classical three-layer architecture [8], where a deliberator forms a plan, a sequencer turns it into actions, and a reactive controller runs primitives; S2, S1, and S0 sit at those three altitudes. That template runs one deliberator and one sequencer over a single task, so arbitrating the body stays implicit in the sequencer. TypeGo instantiates a planning stack per task: an S2/S1 pair inside a *process* with its own control block and lifecycle. Multiple tasks then plan at once, and the body becomes a contended resource, so actuator arbitration, preemption, and source-dependent resumption become explicit runtime services (§5). Driving these layers with LLMs also removes a property the template relied on: a deterministic sequencer that ran in milliseconds, leaving only the deliberator slow. TypeGo’s sequencer is itself an LLM call, seconds long and stochastic, so fast sequencing must be recovered by other means: speculative skill streaming (§6) and a compiled S0 that never calls a model. Inspired by dual-process theory [9], some recent planners pair a fast “System 1” with a slow “System 2” and route between them [10–13]. Such two-tier designs activate only one tier at a time and switch between modes. In contrast, TypeGo runs its four loops concurrently across timescales (S0–S3), rather than as alternatives selected per request.

LLM-based robot planning. LLMs are widely used to turn instructions into robot plans [14, 15]. Existing systems are either *monolithic planners* that prompt once for a full plan (Code-as-Policies [2], TypeFly [1]) or *step-by-step planners* that query per action (SayCan [3], Inner Monologue [16], ReAct [17]). A monolithic plan delays the first action and cannot recover when the scene changes; a step-by-step planner reacts but stalls the robot between actions while it rebuilds its prompt. TypeGo explores a middle point: asynchronous LLM loops across levels of abstraction, plus a speculative planner (S1) that aims to hide planning latency at runtime. Orthogonally, end-to-end vision–language–action models map perception directly to low-level control [18]; such a policy can sit *inside* a TypeGo skill, while TypeGo addresses concurrency, preemption, and latency-hiding above the policy level.

Multi-agent LLM systems. Prior multi-agent work arranges agents in chains or graphs: some refine an instruction top-down [19, 20], while Mixture-of-Agents [21] combines outputs from multiple proposer models through an aggregator. In these designs agents run sequentially, causing execution time for a single request to grow with the number of agents. TypeGo instead runs each planner in

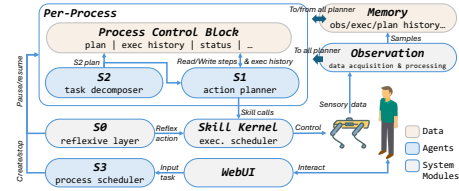


Figure 1: TypeGo runtime. A hierarchy of agents (blue) includes the global S3 (process scheduler) and S0 (reflexive layer), and the process-scoped S2 (task decomposer) and S1 (action planner). S3 manages the process lifecycle based on user input or its own decisions. S2 and S1 together turn a single task into a continuous stream of skill calls. The Skill Kernel arbitrates and schedules skill calls issued by multiple processes. Observation taps the robot body’s sensor stream, and Memory periodically samples the agent’s state into a retrieval-accessible store; both are available system-wide.

its own loop, coordinated through synchronization and scheduling rather than sequential dependency, keeping planning latency low.

Operating systems for LLM agents. A recent line of work borrows OS structure for software agents. AIOS [22] adds an “LLM kernel” that schedules and manages agent requests, and MemGPT [23] treats the context window like virtual memory, paging information in and out. TypeGo shares their OS framing but targets a fundamentally different workload: a multi-tasking *embodied* agent, whose resources include physical actuators with exclusivity and safety constraints rather than context tokens or API calls.

3 System Overview

TypeGo realizes the runtime thesis (§1) as a stack of asynchronous planning loops (S0 through S3) over a *Skill Kernel* that mediates the embodied agent’s physical actuators (Figure 1).

The runtime borrows its structure from OS design (Table 1), which we use as a guiding analogy rather than decoration: most components have an OS counterpart, and we are explicit about where it deliberately departs. A user-authored *task* is a static, natural-language job specification, analogous to an OS program; the runtime realizes each task as a *process* with its own process control block (PCB). The *Skill Kernel* plays the role of the OS kernel, arbitrating the robot body’s typed actuator subsystems (its devices) among concurrent processes; *reflexes* act as interrupt handlers with priority preemption; and the speculative *S1 step queue* resembles a read-ahead buffer that decouples planning latency from execution.

Where the analogy departs. The OS framing is productive precisely because an embodied agent breaks several assumptions a classical OS takes for granted, and these departures are where the research problems lie. First, *resources are physical and semantic at once*: preempting an actuator has real-world, sometimes unsafe, consequences, so clean preemption is a correctness property enforced by bounded-interruption skills. Second, *scheduling requires understanding meaning*: S3 decides which process runs from an LLM’s reading of the current observation and behavioral guidelines, a semantic policy a CPU scheduler never needs; additionally, whether a preempted process later resumes depends on the *source* of the preemption (§5.2). Third, *programs are natural language*: prescriptions are ambiguous and are compiled or interpreted at

Table 1: OS analogues used throughout the paper.

TypeGo concept	OS analogue
Task (natural-language spec)	Program (job specification)
Process (handles one task)	Process
PCB (planning state)	Process control block
Skill Kernel	OS kernel (resource manager)
Subsystem	Device with exclusivity policy
Skill	Driver call
Reflex (S0)	Interrupt handler
S1 step queue	Read-ahead buffer
Source-dependent resumption (S3)	Process scheduler policy
Memory (future work)	File system / virtual memory

runtime by LLMs rather than ahead of time (§4). Fourth, *correctness is probabilistic*: any planning layer can be wrong, so the runtime must contain and recover from bad outputs rather than assume correct programs. We return to these departures throughout.

The three mechanisms behind this runtime map to the sections that follow: *speculative skill streaming* overlaps planning with execution to hide LLM latency (§6); the Skill Kernel and S3 scheduler together multiplex concurrent tasks, running non-contending processes in parallel and preempting the rest cleanly (§5); and *prescriptions* make natural language a first-class programming medium, routed to the online planners or compiled into S0 interrupt handlers. Users and developers interact with TypeGo through three programming abstractions: *prescriptions*, *observations*, and *skills*.

4 Programming Abstractions

TypeGo exposes three programming abstractions: user writes *prescriptions*, and system developers supply *observations* and *skills*.

A **prescription** is TypeGo’s natural-language programming interface: it lets a user shape how the embodied agent behaves without writing code. The key design point is that TypeGo routes each prescription to the layer whose latency matches its purpose; the user declares which kind they are writing, so routing is fixed at authoring time rather than inferred at runtime. TypeGo supports three kinds. *Tasks* are high-level objectives (e.g., “patrol the hallway to find a soccer ball”) consumed *online* by the S1–S3 planners; each task spawns its own process, and multiple tasks may coexist. A *persona* states how the agent should behave in general (e.g., “cautious and quiet indoors”); an LLM compiles it *offline* into per-layer guidelines that bias S1–S3 planning, so one description shapes every task rather than a single one; we do not evaluate this path here. *Reflexes* are natural-language condition–action pairs that an LLM compiles *offline* into a pair of Python functions over authored skills and observation accessors; S0 evaluates them at high frequency for low-latency reactions (e.g., “when anything is closer than 0.2 m in front, step back 0.5 m”). S0 has the highest priority in TypeGo: a firing reflex interrupts any in-progress process holding contended subsystems and resumes it on completion.

An **observation** is a structured, continuously updated representation of the embodied agent’s environment and internal state, grounded in the robot body’s sensors and exposed to all planners. It is a collection of `fields`, each with a developer-specified semantic role, data source, post-processing, and serialization. For example,

```
@robot_skill("name", desc="...", subsystem=SubSystem.MOVEMENT)
def name(arg: type, [pause_evt], [stop_evt]) -> SkillReturn:
    # skill logic; check events for preemption
```

Listing 1: TypeGo skill interface. A decorator declares the skill’s name, description, and subsystem (e.g., `MOVEMENT`, `SOUND`, `DEFAULT`). A skill may optionally accept `pause_evt` and `stop_evt` handles for interruption; skills that omit them are atomic. It returns a `SkillReturn` carrying a success flag and message.

an `object_detections` field binds to the front RGB camera, applies YOLO, and serializes detections as JSON for planner prompts.

Skills are callable primitives that implement concrete operations on the robot body; developers author and verify every skill (Listing 1). Three principles guide their design. *Synchronous*: a skill blocks until it returns, matching ordinary function-call semantics, though TypeGo invokes it asynchronously through the Skill Kernel (§5.1). *Interruptible or atomic*: interruptible skills weave `pause_evt` and `stop_evt` into their control loop and stop within a bounded time t_s , returning a `SkillReturn` that distinguishes success, failure, and interruption; a few hardware-constrained skills (e.g., `stand`) are atomic. This bounded-interruption guarantee is what lets TypeGo recover from failures and switch tasks with bounded delay. *Authored, not synthesized*: a skill’s control flow is fixed by the developer and never synthesized at runtime by TypeGo’s agents, though it may call learned models internally, keeping core operations predictable and verifiable.

Most skill interfaces are platform-agnostic, so TypeGo’s planners operate largely independently of the underlying hardware. Composite skills may encapsulate an entire optimized control procedure in a single invocation, provided they honor preemption within t_s .

5 Runtime Abstractions

TypeGo’s runtime is structured like a small operating system for an embodied agent. Two abstractions carry the OS analogy: the *Skill Kernel* (§5.1), a resource manager that arbitrates typed actuator subsystems of the robot body among concurrent processes, and the *process* (§5.2), a schedulable, stateful unit that handles one user task. Together they meet two design objectives: (1) decouple skill design from scheduling, giving developers an extensible interface for adding new capabilities without touching the core runtime; and (2) enable safe concurrent and interruptible execution across multiple tasks, so that processes with disjoint exclusive subsystems run in parallel while conflicting ones are serialized or preempted.

5.1 Skill Kernel

The Skill Kernel is TypeGo’s resource manager. Like an OS kernel mediating device access among processes, it mediates a typed registry of actuator *subsystems* (e.g., locomotion, audio) among the processes. It enforces two properties: across processes, two skill calls cannot conflict over a shared physical resource; within a process, a new skill call may override an in-flight one. The kernel arbitrates by acquiring subsystems on behalf of *skills*, the analogue of driver calls.

Subsystems. A *subsystem* abstracts a physical resource of the robot body that a skill consumes during execution. Each is configured

along two axes: *exclusive_per_process* (only one process may hold it at a time) and *parallel* (multiple invocations may run concurrently). Developers can add a new subsystem (e.g., a manipulator arm or a display) without modifying the scheduler or existing skills. TypeGo ships three: **MOVEMENT** (exclusive, serial) for locomotion and gestures, which rejects a second requester to prevent conflicting motion; **SOUND** (shared, serial) for audio output, which serializes invocations in a queue; and **DEFAULT** (shared, parallel) for skills needing no exclusive hardware.

Execution interface. When a skill such as `move_forward` is invoked, its declared subsystem (**MOVEMENT**) is marked held by the calling process, blocking other processes that need the same exclusive subsystem. Processes use the non-blocking `execute` interface, which returns immediately with a unique id and a flag indicating whether the skill was scheduled or rejected due to a conflict; the caller queries progress via the id.

5.2 Process Abstraction

A *process* is TypeGo’s unit of multitasking: the runtime’s live representation of one task, carrying everything needed to plan it, pause it, and resume it. It is a runtime abstraction of TypeGo’s own, not an operating-system entity; the OS schedules neither its planning loops nor its skill calls, and its program is a natural-language task rather than compiled code. What it borrows from an OS process is structure: a private control block, a lifecycle the scheduler drives, exclusive claims on shared resources, and preemption with defined resumption semantics. The runtime realizes each task by spawning a process with a private process control block (PCB) which holds the process *ID*; the *task* text (user- or S3-generated); the S2/S1 *plan* (subtask list and index, global events, S1 step queue); the skill-execution *history*; the lifecycle *status* (IN-PROGRESS, PAUSED, SUCCESS, FAILED); and the robot skill set.

Life cycle. For every incoming task, S3 instantiates a PCB and a dedicated S1/S2 pair. While IN-PROGRESS, S2 decomposes the task and S1 streams skill calls; each call attempts to acquire its required subsystems, and any failure moves the process to PAUSED, where both loops block until S3 reschedules it. The process ends in a terminal state when it completes or is stopped by S3 or the user.

Process types. TypeGo distinguishes processes by *source*: **user** processes carry explicit instructions; **S3-initiated** processes are spawned by S3 when standing behavioral guidelines fire; and a single **idle** process runs default behavior whenever nothing else is active.

Scheduling: concurrency. Two processes run simultaneously only when the exclusive subsystems they hold are disjoint; otherwise, one is PAUSED until the other releases its subsystems. A process retains its subsystems until it pauses or terminates; it keeps **MOVEMENT** reserved even after `move_forward` finishes, because a following `take_pic` may depend on the physical body staying still.

Scheduling: source-dependent resumption. Preemption raises two questions: which process runs now, and what becomes of the one that yields. A priority scheme answers the first; TypeGo answers the second explicitly, with two outcomes. Service managers declare the same distinction statically: an Erlang/OTP supervisor marks each child permanent, transient, or temporary [24], and a systemd unit sets `Restart=` [25]. TypeGo derives it per preemption instead.

```
# S2 output for "Find an apple and report to the user. Take a picture whenever
you see a ball".
{"operation": "update",
 "subtasks": [
  {"desc": "Find the apple", "criteria": "Found, or nowhere left to search"},
  {"desc": "Report to the user", "criteria": "See and speak to user"}],
 "global_evt": [{"cond": "see a ball", "resp": "take a picture"}]}
```

Listing 2: S2 output for a sample task, decomposed into two subtasks and one global event.

Interrupt-and-return fires when a transient event (e.g., a person walks into view) triggers a brief detour: the new process takes over, completes, and the displaced process auto-resumes. *Replace-without-return* fires when a new user instruction supersedes the old goal for good: the previous process is stopped and never auto-resumed. S3 derives which outcome applies from the preemption’s source: a **user** process replaces-without-return (S3 stops every running user and S3-initiated process, none auto-resumed unless the user’s task explicitly asks for it); an **S3-initiated** process always interrupts-and-returns (it preempts only user or idle processes, and S3 auto-resumes the predecessor when it terminates); the **idle** process never preempts.

6 Hierarchical Collaborative Agents

TypeGo pairs rapid response with deliberation through four asynchronous agents, from S3 (slowest, most abstract) down to S0 (fastest, most concrete). The names generalize the System 1 / System 2 dichotomy [9] to four cadences, but, unlike that two-tier view, our tiers run *concurrently* rather than as alternatives selected per request. S3 and S0 are global across processes; S2 and S1 are instantiated per process. Each runs its own independent loop.

System 3: process scheduler. S3 manages processes from two inputs: explicit user tasks and behavioral guidelines embedded in its prompt. The second lets TypeGo act autonomously. For example, under the guideline “you may pause a non-emergency task to engage a nearby person,” S3 pauses “find a sports ball” and starts “play with the person” when it detects someone. User-initiated processes outrank S3-initiated ones; S3’s concurrency and preemption policy is detailed in §5.2.

System 2: task decomposer. S2 decomposes a task into (1) an ordered list of *subtasks*, each a natural-language description of what to achieve plus a success criterion, which S1 consumes one at a time; and (2) a set of *global events*, condition/response pairs that S1 monitors as reactive interrupts during execution (Listing 2). After the initial decomposition, S2 periodically (every 2 s in our implementation) re-inspects the action history and observation and decides whether to keep or update the decomposition.

System 1: action planner. S1 turns the current subtask into a stream of skill calls (Listing 3) via two concurrent threads: a *streamer* that generates steps and an *executor* that runs them, both able to call an LLM. Rather than emit all steps at once, the streamer generates a few per LLM call into a bounded queue (size three in our case), while the executor dequeues the oldest; each dequeue triggers one enqueue, so generation overlaps execution. The stream-interpretation idea itself follows TypeFly [1]; TypeGo’s contribution is to embed it in a multi-cadence runtime, where S2 continuously

```
// In-progress skill: stand_up(), Subtask: Find the apple.
{
  "steps": [
    {
      "branches": [{"action": "search('apple')", "cond": "always"}],
      "branches": [
        {"action": "approach('apple')", "cond": "visible('apple')"},
        {"action": "goto_waypoint(3)", "cond": "always"}
      ]
    },
    {
      "subtask_complete": false
    }
  ]
}
```

Listing 3: S1 streamer output for a sample subtask. While `stand_up` runs, the streamer speculatively generates two further steps: search for the apple, and then approach it if visible or move on otherwise.

revises the subtask that S1 streams against and S3 can preempt the whole loop, rather than running it as a single open-loop interpreter. Each *step* holds one or more mutually exclusive *branches* pairing a condition (a predicate over observation or a short natural-language expression) with a skill call; the executor invokes the matching branch. The executor also watches S2’s global events: when one fires, it interrupts the in-flight skill, records the event on the PCB, and the streamer handles it on its next cycle.

System 1 Prime (S1P). Streaming hides planning latency only once the queue is filling; the first step still waits on an LLM call. S1P, a subcomponent of S1, closes this gap: at process creation it emits one immediate skill call from a fast LLM (Groq-hosted GPT-OSS-120B in our case) without chain-of-thought, preferring visible actions such as movements or gestures.

System 0: reflexive layer. S0 evaluates a list of condition functions and invokes the matching action function; both are plain Python that an LLM compiles from reflexes offline, so no LLM sits on the reaction path. S0 outranks every task-driven process: a firing reflex pauses every process holding a contended subsystem and resumes each on completion. Reflexes themselves preempt one another by hard priority.

7 Implementation

Our current prototype is implemented in Python and packaged as a ROS2 node, with three parts: the TypeGo runtime (planning and execution), a lightweight platform-specific SDK that exposes skills and sensor data over ROS2, and a pool of AI services (LLMs and perception models). S0 runs at 100 Hz, S2 and S3 at 0.5 Hz, and S1 is event-driven. A process is a PCB record plus Python threads inside that node: an S2 loop and S1’s streamer/executor pair per process.

Platform SDK and case study (Kalos). The ROS2 SDK is TypeGo’s data and control provider for the robot body: it publishes sensor data (RGB images, LiDAR scans, state estimates) and forwards control commands over TCP, and integrates SLAM and navigation with an indexed waypoint service. Our initial deployment, Kalos, runs TypeGo on a Unitree Go2 and registers skills in four categories: basic locomotion (`move_forward`, `rotate`, `nav`); postural and expressive actions (`stand`, `sit`, `nod`); high-level composites (`search`, `approach`, `follow`, `goto_waypoint`, `patrol`); and system utilities (`take_pic`, `speak`, `query_memory`). TypeGo is not tied to the Go2: it runs on platforms as modest as the Peto quadruped [26] or a wheeled robot, requiring only basic locomotion and continuous perception.

AI services and deployment. An edge server (with an RTX 4090) runs an HTTP gateway that forwards LLM calls to the cloud and dispatches perception to a pool of stateless gRPC workers: object

detection (YOLO [27]), open-vocabulary detection (OmDet [28]), image-text embedding (CLIP [29]), and text-to-speech. Our deployment uses two off-robot machines over Wi-Fi: a laptop for the runtime and a GPU edge server, but the stack can be consolidated onto a single on-robot edge device (e.g., an NVIDIA Jetson) for self-contained mobile operation.

8 Preliminary Evaluation

We use a small prototype task suite to validate three claims from §1: **effectiveness** (can the prototype complete composite, long-horizon, perception-grounded tasks end-to-end, and degrade gracefully when it cannot?); **responsiveness** (does layered planning reduce per-step latency relative to a re-plan-every-step controller while retaining more adaptivity than a monolithic planner?); and **concurrency** (can the runtime admit concurrent tasks or reflexes, how fast is preemption, and at what cost?). The goal is not to exhaustively benchmark robot planning, but to test whether the proposed OS-style abstractions behave as intended on a real embodiment: effectiveness probes the end-to-end runtime, responsiveness probes multi-cadence planning with speculative streaming, and concurrency probes the Skill Kernel together with scheduling.

Baselines and setup. Our two baselines, *ReAct* [17] and *plan-and-execute* (PAE), share TypeGo’s skill library and use minimal-reasoning prompts. TypeGo and both baselines drive every planning layer with GPT-5.4; the sole exception is TypeGo’s S1P fast-response path, which uses Groq-hosted GPT-OSS-120B (§6). *ReAct* emits one reasoning step plus one skill call per step from the instruction, current observation, and prior trace; it has no decomposition/execution split, speculation, reflex, or scheduler, so it cannot plan its next action until the current skill returns. PAE prompts once for a complete ordered plan, executed open-loop. We evaluate eight tasks in three categories (Table 2): explicit commands (T1–T3), open-ended planning (T4–T5), and scheduling and concurrency (T6–T8), with 10 trials per task.

8.1 Effectiveness and Responsiveness

Table 3 reports success and three latency metrics: *TTFA* (input to first action), *total* (normalized end-to-end time), and *per-step* (wait between skills), plus normalized token usage. In this prototype task suite, TypeGo attains the lowest TTFA (40% below *ReAct*, 73% below PAE) and the lowest completion time, and cuts per-step delay by 50% relative to *ReAct*. Two design choices drive this. First, S1P emits one immediate skill call at process creation, overlapping the first action with the higher-level planners’ deliberation and pulling reaction latency under one second; this action matches the eventual plan in roughly 60% of cases and consistently produces visible feedback within the first second. Second, the speculative step queue keeps upcoming skill calls ready, removing the per-step planning stall that dominates *ReAct*’s latency. The cost is token usage: TypeGo’s multiple asynchronous loops consume substantially more tokens than *ReAct* (one loop) or PAE (one initial call), an explicit responsiveness-cost tradeoff we leave to future runtime-policy optimization.

Failure cases. T3 has a blocked path: TypeGo and *ReAct* both recognize the failure and report it after a few attempts, while PAE

Table 2: Evaluation tasks for TypeGo on Kalos. T1–T5 run standalone; T6–T8 each inject a second activity while T5 is already running.

ID	Category	Task Description
T1	Explicit Commands	Turn around and face the person.
T2		Step back, turn left twice, and nod three times.
T3		Go to the hallway (path blocked; the system should report failure).
T4	Open-ended Planning	Look for a chair in the hallway and take a picture of it.
T5		Look around for a sports ball. If you find one, walk up to it and bark twice. If you’ve checked the whole area and there’s none, return to the start and sit down.
T6	Schedule / Concurrency	While running T5, the user issues “stop, check what’s on your left, then resume”; the embodied agent must abort safely and then continue.
T7		While running T5, ask the embodied agent how many places it has explored.
T8		While running T5, a person suddenly appears in front of the robot and trips S0’s obstacle-avoidance reflex once.

Table 3: Success count and latency breakdown across planners (T1–T5); total completion time and token usage are normalized before averaging. The results suggest that the runtime can reduce reaction and per-step delay relative to ReAct, at the cost of higher token usage.

Planner	Success (/10) ↑					Avg. Time (mean/std) ↓			Tokens ↓
	T1	T2	T3	T4	T5	TTFA (s)	Total (Norm.)	Step (s)	Norm.
TypeGo	10	10	8	6	10	0.87/0.05	1.00/0.12	0.62/0.54	1.00/0.33
ReAct	10	10	7	6	9	1.46/0.23	1.33/0.09	1.25/0.27	0.32/0.08
PAE	10	10	0	9	10	3.27/1.73	1.14/0.26	0.01/0.01	0.05/0.02

Table 4: Scheduling overhead (base task T5). *Add-on* is the concurrent task’s standalone duration; *Total* is both tasks together under TypeGo’s scheduling. *Overhead* is the runtime beyond running the activities independently: for the serialized path (T6) it splits into S1 step-queue replan plus scheduling cost (1.20 + 0.22); for the parallel path (T7) the add-on overlaps the base, so overhead is compared with the base alone; T8 is the one-shot S0 reflex detour.

Task	Base (s)	Add-on (s)	Total (s)	Overhead (s)
T6		6.17/1.58	42.35/2.37	1.20 + 0.22
T7	34.76/2.16	4.37/1.21	34.91/1.94	0.15
T8		0.50/0.01	35.31/2.13	0.05

has no closed-loop feedback and stalls indefinitely (0/10). This supports the need for reactive re-planning, but also shows why the prototype should be tested on richer failure modes. T4 is the only task where PAE wins: its global plan commits to going to the hallway before searching, whereas TypeGo and ReAct greedily inspect chairs already in view. It is a known bias of step-by-step planning toward locally promising actions [30, 31] and an open design point for the S1/S2 split.

8.2 Concurrency and Scheduling

Each concurrency task injects a second activity during T5, exercising a different scheduling path and, with it, a different facet of the abstractions. T6 and T8 both follow the *interrupt-and-return* path (one triggered by a user instruction, one by an S0 reflex) and qualitatively confirm that the displaced process is paused and correctly resumed afterward, while T7 confirms that processes holding disjoint subsystems run in parallel. Since neither baseline can admit a second activity mid-execution, we report only TypeGo’s scheduling overhead, the runtime beyond running the activities independently: their summed standalone durations when they contend and serialize (T6), or the base alone when the add-on runs in parallel (T7). Overhead tracks subsystem contention (Table 4). In T6, the injected instruction contends with T5 for the MOVEMENT subsystem, so S3 serializes the two, pausing and resuming T5 at a total cost of 1.42 s. Scheduling itself accounts for 0.22 s of this; the remaining 1.20 s

is S1 replanning: an S3 preemption switches between two major tasks, so the queued steps no longer match the situation the process returns to and S1 must refill its step queue. T7’s memory query uses no movement subsystem and runs in parallel (0.15 s). T8’s S0 reflex instead handles a brief emergency without changing the running task, so the plan stays valid, no replanning is needed, and the detour costs 0.05 s. These early results suggest that runtime overhead is tied to subsystem contention and to whether the task itself changes, not merely to the presence of a second activity.

9 Future Work: Toward a Full Agent OS

TypeGo is an early step toward an operating system for embodied agents, not a finished OS. Our prototype focuses on three primitives that were immediately necessary for validating the idea on a physical embodiment: concurrent processes, resource arbitration over shared actuators, and multi-cadence planning. A complete agent OS needs additional primitives that make long-lived, adaptive agents manageable across hours, days, and deployments.

Memory as an OS primitive. Memory remains the most important area for future work. Today, TypeGo includes only a simple memory service: the runtime samples observations and execution history, stores them in a retrieval-accessible database, and exposes retrieval through skills such as `query_memory`. This works, but it treats memory as an application service rather than an OS abstraction. In an agent OS, memory should be a first-class managed resource, analogous to a file system or virtual memory: processes should name, share, isolate, evict, checkpoint, and recover state through a common interface.

We envision a memory hierarchy mirroring the planners’ cadence, so each layer reads state at the abstraction and frequency it can afford: **M0**, raw structured observations (detections, pose, timestamps) consumed by the fast S0/S1 loops; **M1**, natural-language observations distilled from M0 (“at noon, a ball was seen near the kitchen”); **M2**, episode summaries that compress M1 and human interactions for S2; and **M3+**, long-term knowledge such as habits and regularities accumulated across tasks and deployments

for S3’s strategic decisions. Lower tiers update at high frequency near perception; higher tiers are refined by *active consolidation*, asynchronous background processes that group, summarize, and promote entries without stalling real-time control. Crucially, rather than exposing memory only through an explicit query API, the runtime itself would choose which tier to inject into each planner’s context, or even into its KV cache, so memory continuously shapes planning the way virtual memory backs computation.

This raises OS questions that are distinct from ordinary vector-store design. What is the unit of memory: a raw sensor frame, an event, an executed skill, a failed plan, a user preference, or a distilled episode? Which memories are process-private, which are shared across processes, and which survive process termination? How should the runtime arbitrate memory bandwidth and storage budget among competing agents? What consistency guarantees should hold when S1, S2, S3, and S0 read or write different views of the same world state? When should observations be evicted from memory, given that semantically obsolete data may remain safety-relevant? These questions are central to the OS vision because planning, scheduling, and resource control all depend on what the agent can remember.

Richer resource and isolation models. We evaluated TypeGo on a quadruped robot, whose main exclusive resource is locomotion. Other embodiments, such as humanoids and mobile manipulators, will require richer subsystem declarations: arms, hands, gaze, torso posture, displays, tools, private data streams, and external APIs. The Skill Kernel mechanism already supports new subsystems, but future systems should expose stronger isolation policies: which processes may see which observations, which tools may externalize data, and which skills may run concurrently under safety constraints. This moves the Skill Kernel closer to an OS protection boundary, not just a scheduler for actuators.

Cost-aware runtime policy. The current prototype deliberately spends tokens to buy responsiveness. A production agent OS should make this tradeoff explicit and dynamic: S2 monitoring could become event-triggered, S1 could skip re-generation when the queue and observation are unchanged, and smaller distilled models could serve the fast paths. More generally, the scheduler should consider not only actuator contention but also LLM budget, model latency, network availability, and risk.

References

- [1] G. Chen, X. Yu, N. Ling, and L. Zhong, “TypeFly: Low-latency drone planning with large language models,” *IEEE Transactions on Mobile Computing*, 2025.
- [2] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” 2022, arXiv:2209.07753.
- [3] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” in *Conference on Robot Learning (CoRL)*, 2022.
- [4] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “Voxposer: Composable 3d value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, 2023, arXiv:2307.05973.
- [5] S. Yenamandra, A. Ramachandran, K. Yadav, A. S. Wang, M. Khanna, T. Gervet, T.-Y. Yang, V. Jain, A. Clegg, J. M. Turner, Z. Kira, M. Savva, A. X. Chang, D. S. Chaplot, D. Batra, R. Mottaghi, Y. Bisk, and C. Paxton, “Homerobot: Open-vocabulary mobile manipulation,” in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 1975–2011, arXiv:2306.11565.
- [6] P. Liu, Y. Orru, J. Vakili, C. Paxton, N. Shafiuallah, and L. Pinto, “Demonstrating ok-robot: What really matters in integrating open-knowledge models for robotics,” in *Robotics: Science and Systems XX*, 2024.
- [7] R. Shah, A. Yu, Y. Zhu, Y. Zhu, and R. Martín-Martín, “Bumble: Unifying reasoning and acting with vision-language models for building-wide mobile manipulation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, arXiv:2410.06237.
- [8] E. Gat, “On three-layer architectures,” in *Artificial Intelligence and Mobile Robots: Case Studies of Successful Robot Systems*, D. Kortenkamp, R. P. Bonasso, and R. Murphy, Eds. AAAI Press, 1998.
- [9] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.
- [10] S. Saha, A. Prasad, J. C.-Y. Chen, P. Hase, E. Stengel-Eskin, and M. Bansal, “System-1.x: Learning to balance fast and slow planning with language models,” 2025, arXiv:2410.14414.
- [11] A. S. Ziaabari, N. Ghazizadeh, Z. Sourati, F. Karimi-Malekabadi, P. Piray, and M. Dehghani, “Reasoning on a spectrum: Aligning llms to system 1 and system 2 thinking,” 2025, arXiv:2502.12470.
- [12] M. Zhu, Y. Zhu, J. Li, J. Wen, Z. Xu, Z. Che, C. Shen, Y. Peng, D. Liu, F. Feng, and J. Tang, “Language-conditioned robotic manipulation with fast and slow thinking,” 2024, arXiv:2401.04181.
- [13] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori, “Hierarchical reasoning model,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.21734>
- [14] F. Zeng, W. Gan, Y. Wang, N. Liu, and P. S. Yu, “Large language models for robotics: A survey,” 2023, arXiv:2311.07226.
- [15] H. Jeong, H. Lee, C. Kim, and S. Shin, “A survey of robot intelligence with large language models,” *Applied Sciences*, vol. 14, no. 19, p. 8868, 2024.
- [16] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Conference on Robot Learning (CoRL)*, 2022.
- [17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [18] A. Brohan et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning (CoRL)*, 2023.
- [19] B. Prakash, T. Oates, and T. Mohsenin, “LLM augmented hierarchical agents,” in *CoRL 2023 Workshop on Language and Robot Learning (LangRob)*, 2023.
- [20] C. Kienle, B. Alt, O. Arenz, and J. Peters, “Learning hierarchical domain models through environment-grounded interaction,” 2025, arXiv:2505.13497.
- [21] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-agents enhances large language model capabilities,” 2024, arXiv:2406.04692.
- [22] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, “AIOS: LLM agent operating system,” 2024, arXiv:2403.16971.
- [23] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” 2023, arXiv:2310.08560.
- [24] J. Armstrong, “Making reliable distributed systems in the presence of software errors,” Ph.D. dissertation, Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.
- [25] systemd, “systemd.service: Service unit configuration,” Manual page, <https://www.freedesktop.org/software/systemd/man/systemd.service.html>.
- [26] PetoCamp, R. Li, and contributors, “OpenCat-Quadruped-Robot: An open source quadruped robot pet framework,” GitHub, 2021.
- [27] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [28] T. Zhao, P. Liu, and K. Lee, “OmDet: Large-scale vision-language multi-dataset pre-training with multimodal detection network,” *IET Computer Vision*, 2024.
- [29] G. Ilharco, M. Wortsman, R. Wightman, C. Gordon, N. Carlini, R. Taori, A. Dave, V. Shankar, H. Namkoong, J. Miller, H. Hajishirzi, A. Farhadi, and L. Schmidt, “OpenCLIP,” Zenodo, Jul. 2021, doi:10.5281/zenodo.5143773.
- [30] Z. Wang, F. Wu, H. Wang, X. Tang, B. Li, Z. Yin, Y. Ma, Y. Li, W. Sun, X. Chen, and Y. Ye, “Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in llm agents,” 2026, arXiv:2601.22311.
- [31] L. Xu, M. Yu, F. Meng, and J. Zhou, “How far ahead do llms plan? uncovering the latent horizon in chain-of-thought reasoning,” 2026, arXiv:2602.02103.